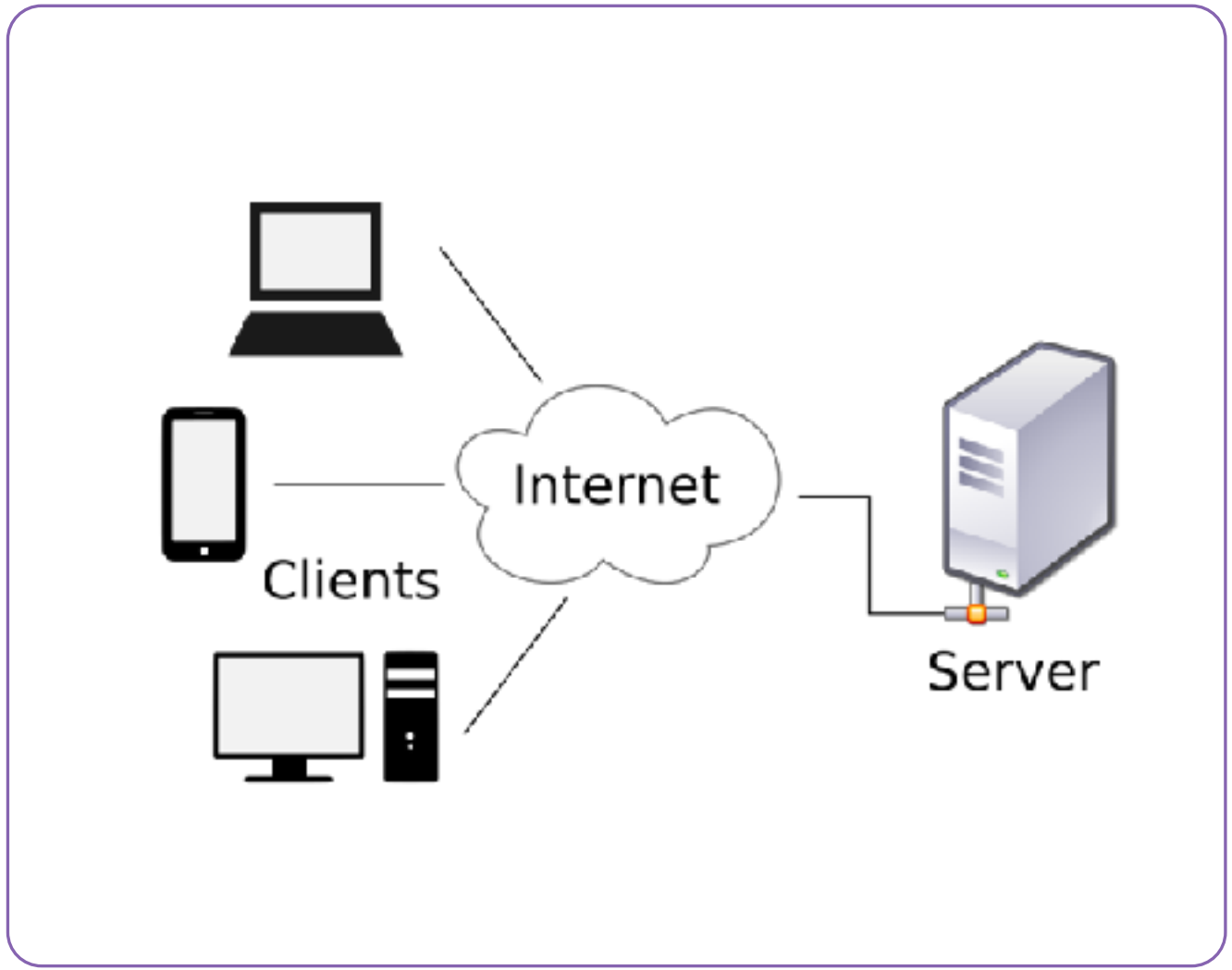
# Browser-to-server communication. HTTP and AJAX

# Browser-to-Server communication

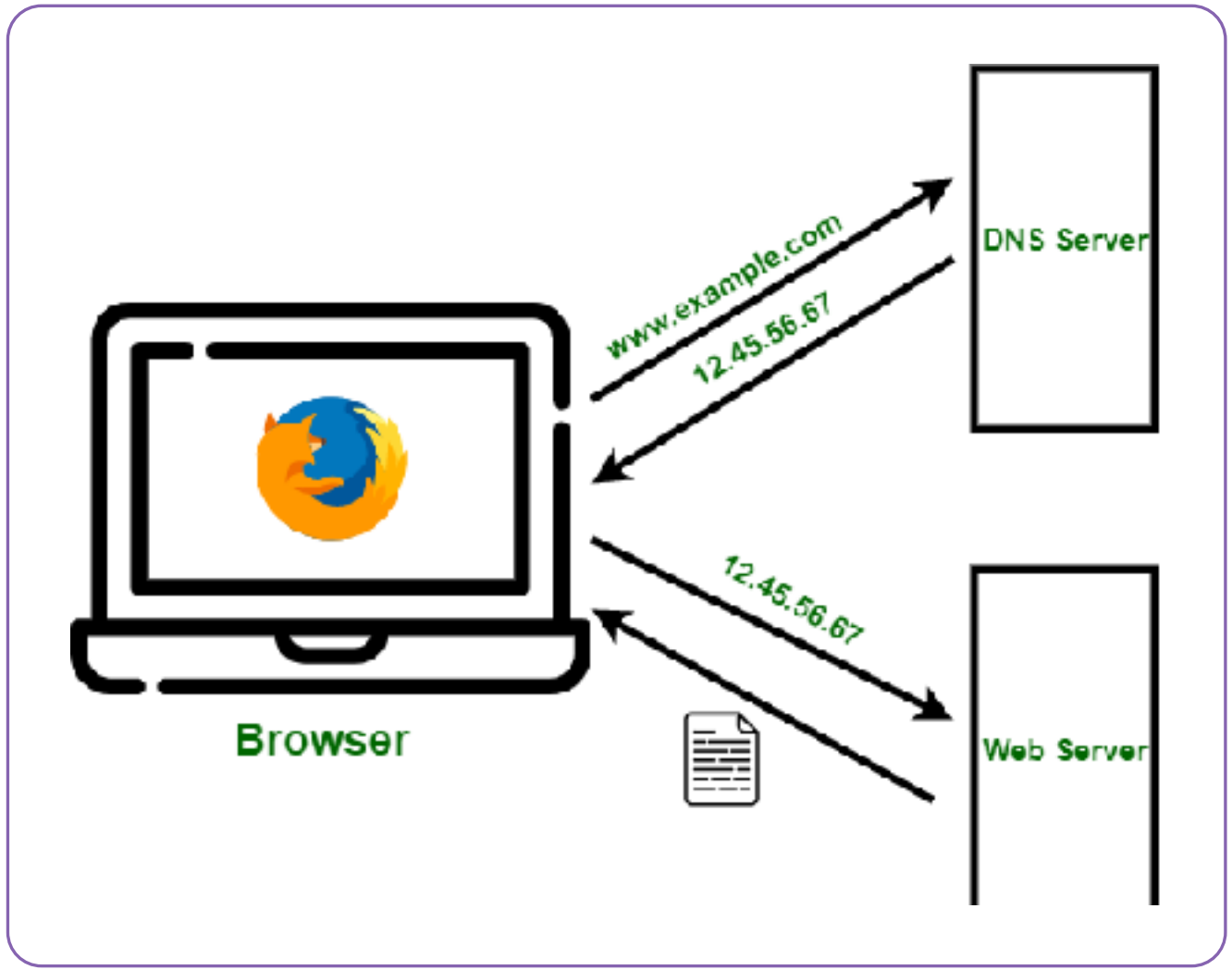
- Represents Client–server model
- Communication via TCP/IP
- Web browsers also rely on DNS to work with URLs



# Domain Name System

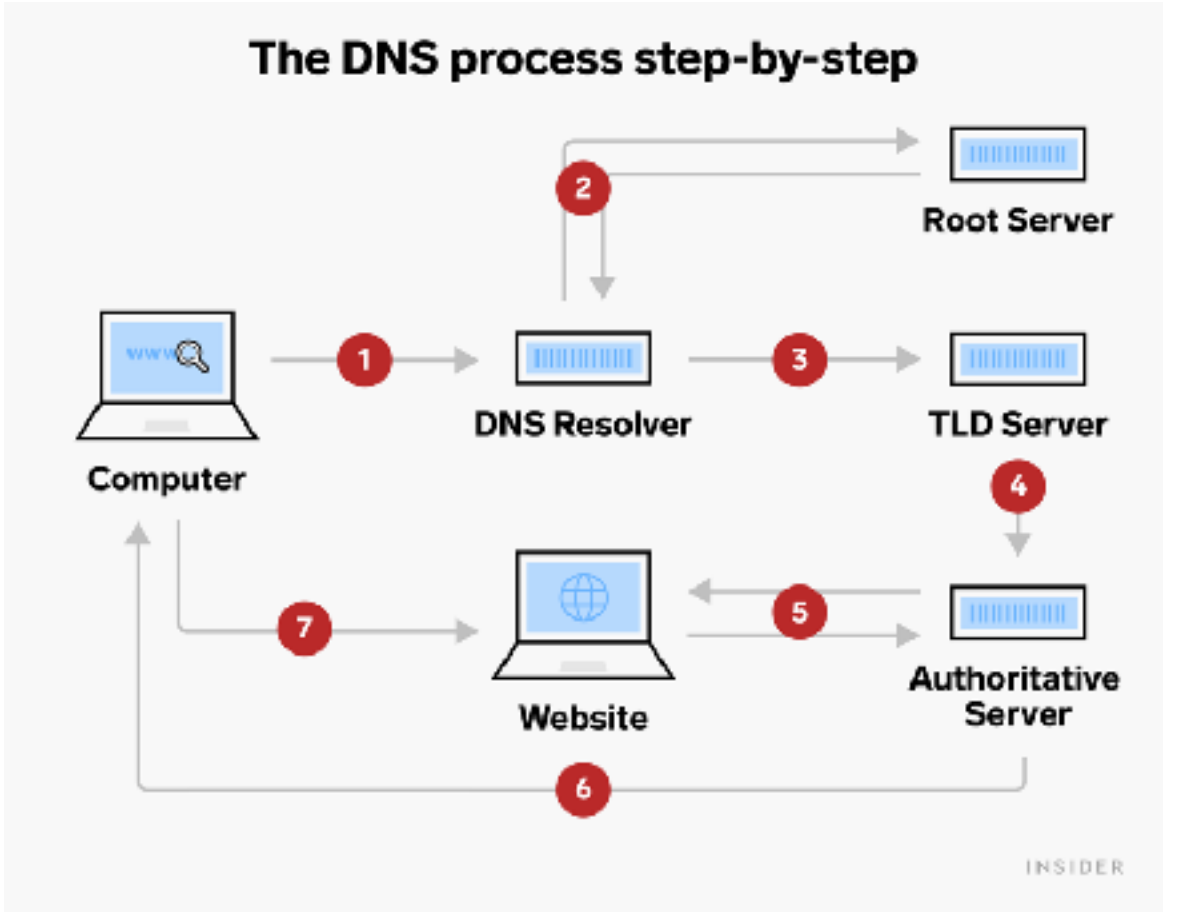
# DNS. Basics

- URL – readable site name for human being to easily remember websites
- IP – digital address of website for computer to operate with
- DNS – process of recognizing the IP address based on URL provided by user



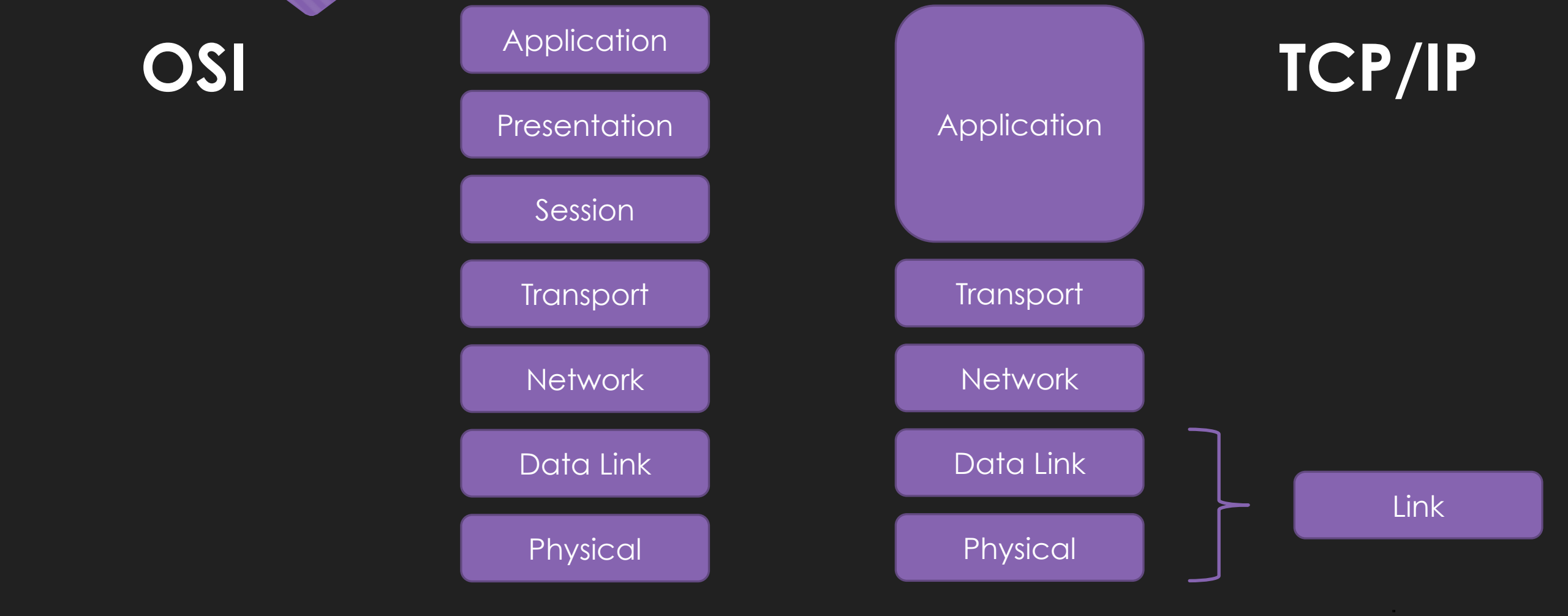
# DNS process

- DNS Resolver contains some cache DNS data
- Root Server – top of DNS hierarchy. It can refer requests to Top Level Domain server
- TLD Server – contains info for domains with specific extension (.com, .net, etc.)
- Authoritative Server – holds DNS records for the domain they serve



# TCP/IP

# OSI Model and TCP/IP model



# TCP/IP. Encapsulation

| | |
|---|---|
| Application | Forms the data that is understandable for application |
| Transport | Adds transport information (source and destination port numbers, etc.) |
| Network | Adds IP header (source and destination IP address) |
| Data Link | Adds header with destination and source MAC address, and trailer for error checking information |
| Physical | Physically transmits information |

# TCP/IP. Protocols

| | |
|---|---|
| Application | HTTP, FTP, SMTP, SSH, etc. |
| Transport | TPC, UDP |
| Network | IP, ICMP, Routers |
| Data Link | Ethernet, ARP, Switches |
| Physical | Cables, Network Interface Cards, etc. |

# Application layer protocols

## A bit more… =)

| Protocol Acronym | Protocol Name |
|---|---|
| BitTorrent | BitTorrent (peer-to-peer file sharing protocol) |
| FTP | File Transfer Protocol |
| HTTP | Hypertext Transfer Protocol |
| HTTPS | Hypertext Transfer Protocol Secure |
| IMAP | Internet Message Access Protocol |
| NTP | Network Time Protocol |
| POP3 | Post Office Protocol (version 3) |
| RDP | Remote Desktop Protocol |
| Skype | Skype (peer-to-peer VoIP protocol) |
| MSNP | Microsoft Notification Protocol |
| SIP | Session Initiation Protocol |
| SMB/CIFS | Server Message Block/Common Internet File System |
| SMTP | Simple Mail Transfer Protocol |
| SSH | Secure Shell |
| TELNET | Terminal Network |
| OpenVPN | OpenVPN (Virtual Private Network) |
| IPsec | IPsec (IP security) |
| PPTP | Point-to-Point Tunneling Protocol |

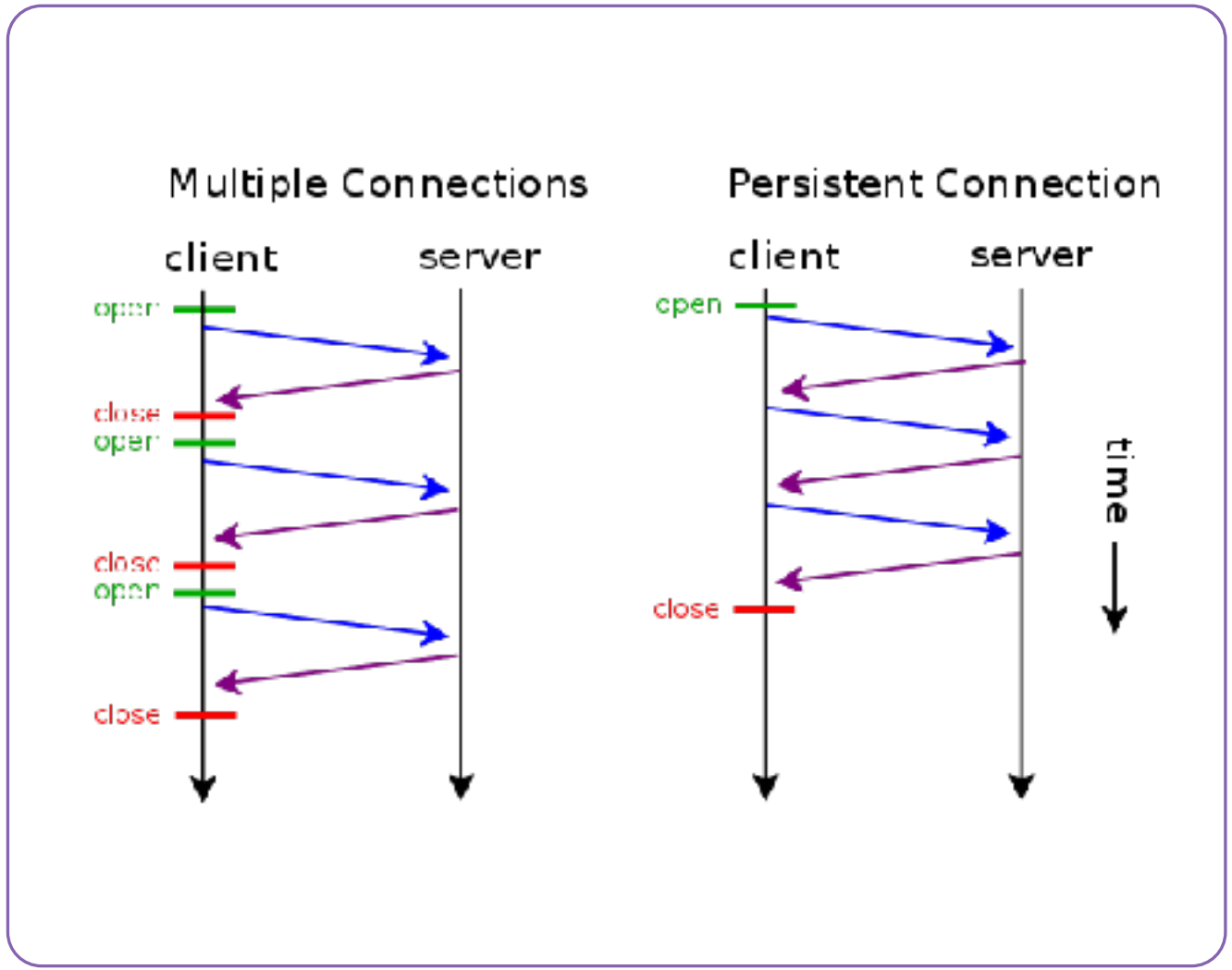


# HTTP

Hypertext Transfer Protocol provides a network protocol standard that web browsers and servers use to communicate. You see HTTP every day because when you visit a website the protocol's written right in the URL

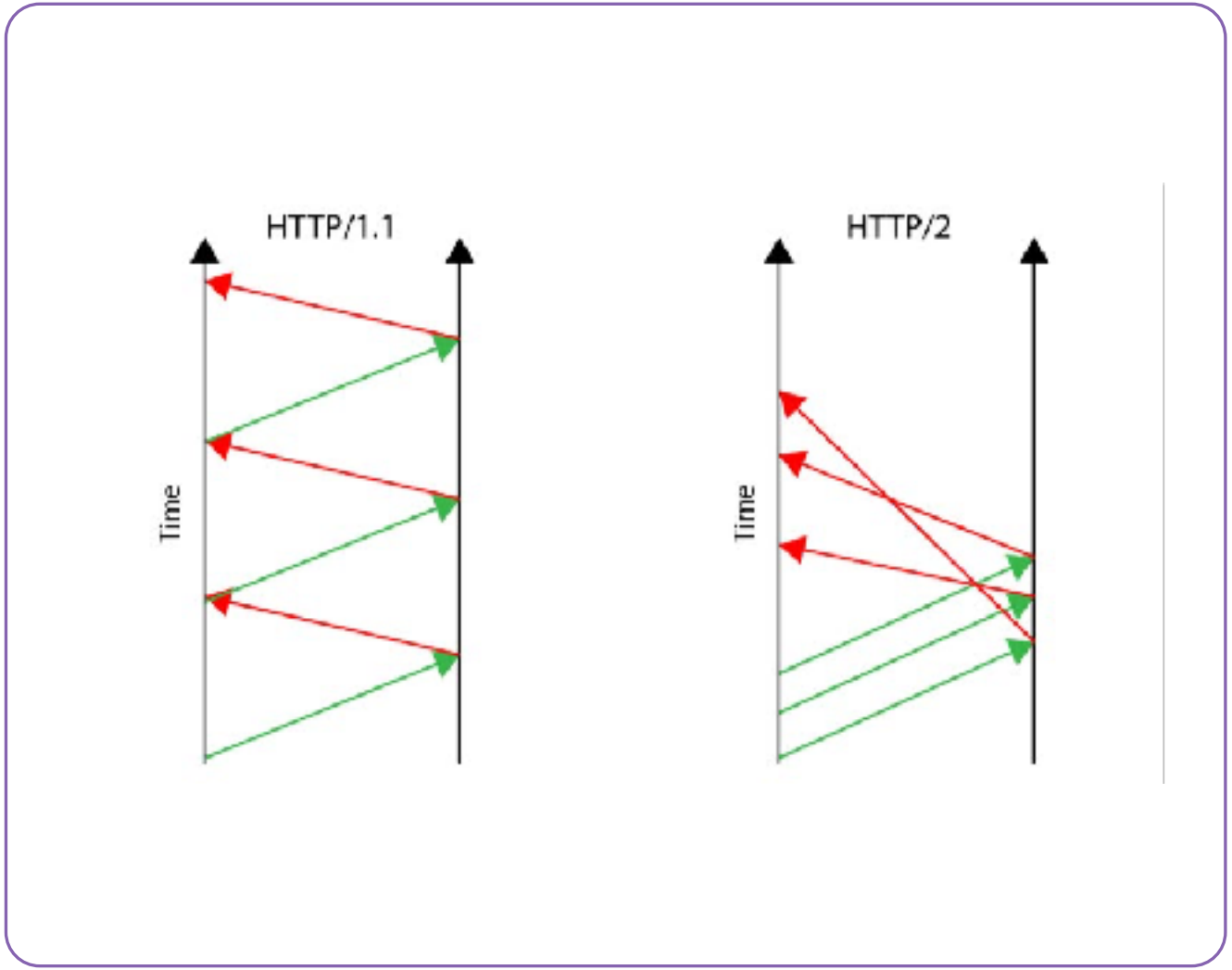
# Keep-alive connection

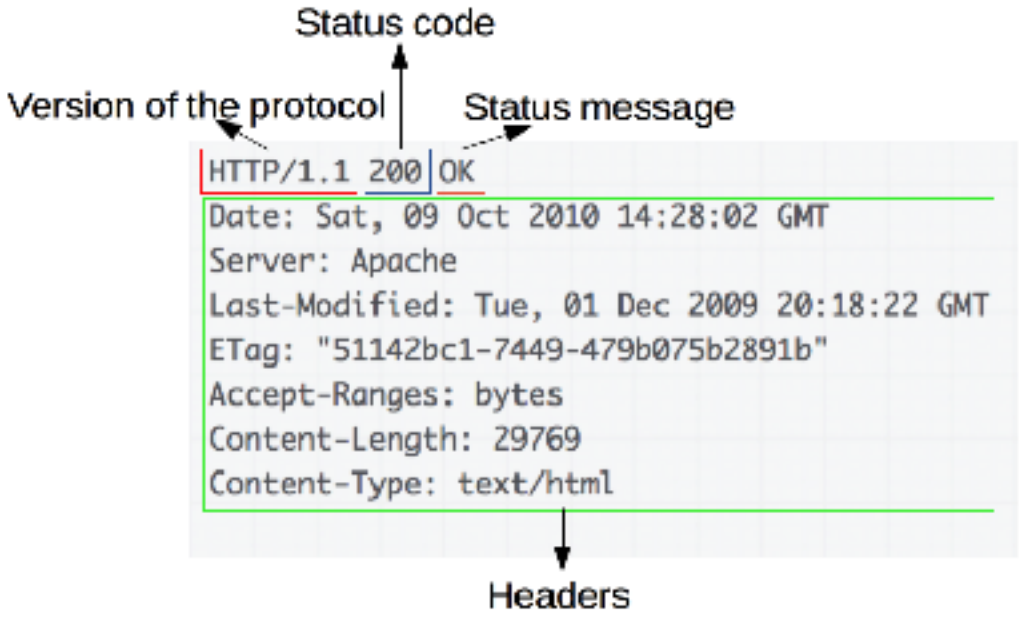
HTTP keep-alive is the idea of using a single TCP connection to send and receive multiple HTTP requests/responses, as opposed to opening a new connection for every single request/response pair.

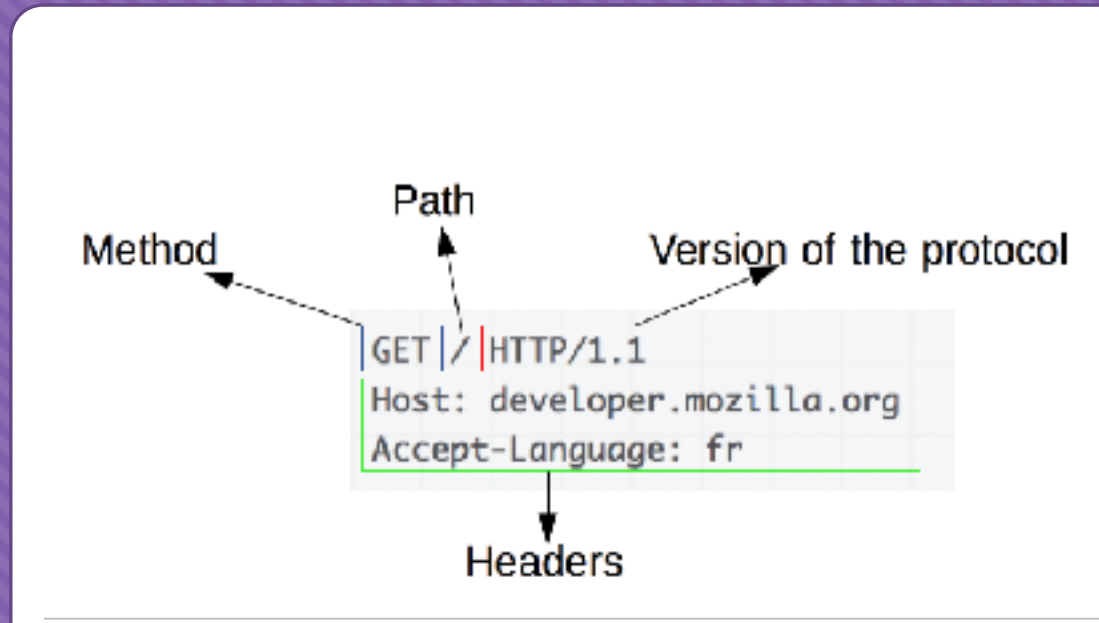


# HTTP. Multiplexing

Multiplexing is a method in HTTP/2 by which multiple HTTP requests can be sent and responses can be received asynchronously via a single TCP connection.






# HTTP. Structure

# HTTP
# Status code

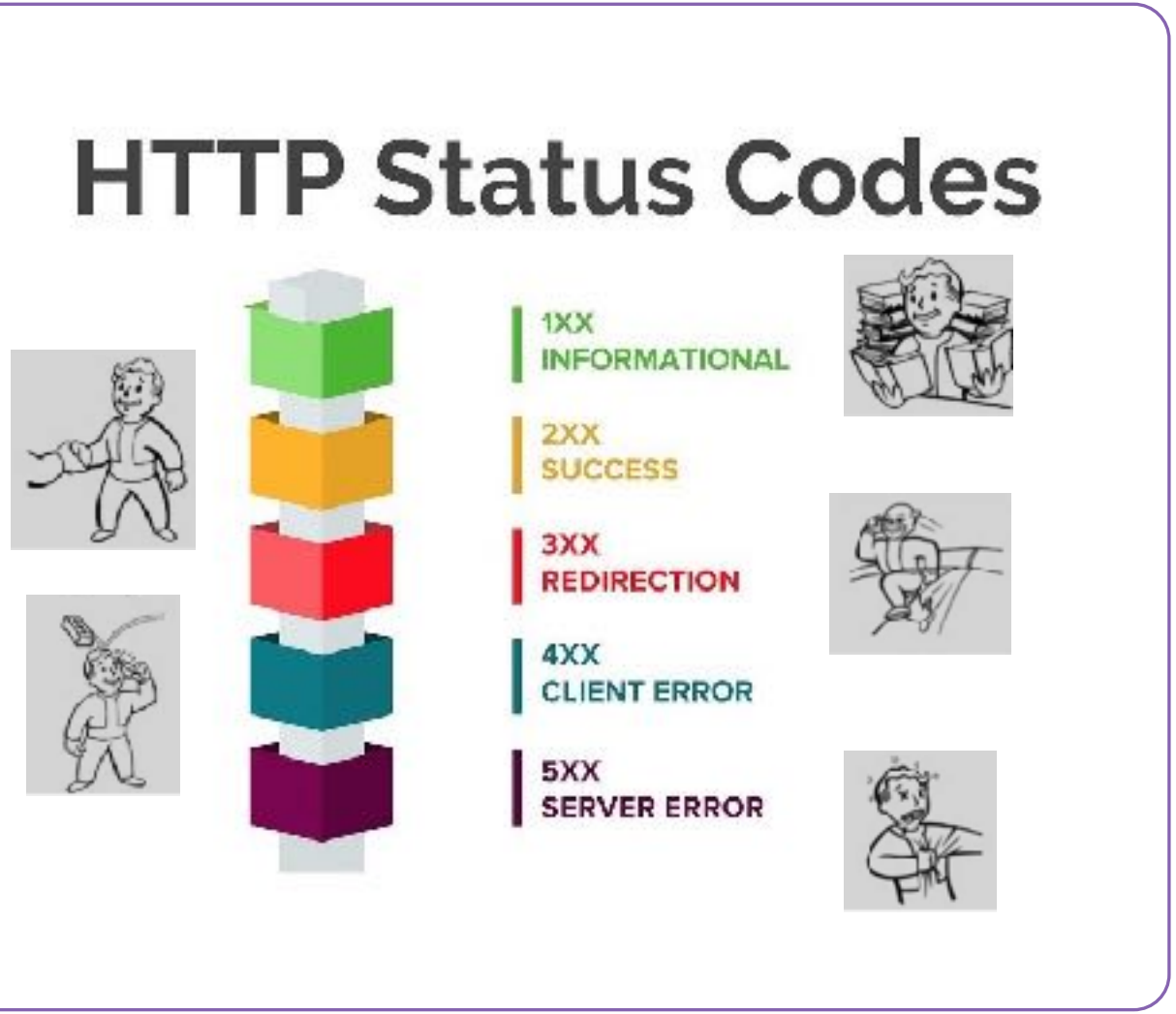
**Info**
- 100 Continue

**Success**
- 200 OK
- 201 Created

**Redirection**
- 300 Multiple Choices
- 304 Not Modified

**Client Error**
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found

**Server Error**
- 500 Internal Server Error
- 501 Not Implemented
- 502 Bad Gateway



# HTTP. Methods

| Method | Description |
| --- | --- |
| **GET** | retrieve data from the specified resource |
| **HEAD** | identical to the GET request but expects response without body. |
| **POST** | requests that the server accept the entity from the request body as a new. |
| **PUT** | requests that the entity be stored under the supplied URI. (create/modify) |
| **PATCH** | applies partial modifications to a resource. |
| **DELETE** | deletes the specified resource. |
| **TRACE** | request changes or additions that have been made by server in received request. |
| **OPTIONS** | returns the HTTP methods that the server supports for the specified URL. |
| **CONNECT** | converts the request connection to a transparent TCP/IP tunnel. (For Proxies) |

# HTTP. Headers

Allow to pass additional information.

- General header
- Request header
- Response header
- Entity header

▼ **Request Headers** view parsed

```
GET /en-US/docs/Web/HTTP/Headers HTTP/1.1
Host: developer.mozilla.org
Connection: keep-alive
Pragma: no-cache
Cache-Control: no-cache
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chro
me/63.0.3239.84 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8
DNT: 1
Referer: https://www.google.com.ua/
Accept-Encoding: gzip, deflate, br
Accept-Language: uk-UA,uk;q=0.9,ru;q=0.8,en-US;q=0.7,en;q=0.6
Cookie: csrftoken=4CDu3wCL1hOfj9solthXtGfQ9ZlFijCa; dwf_section_edit=False; dwf_sg_task_completio
n=False; _ga=GA1.2.1806030116.1485694852; _gid=GA1.2.1630544436.1512942644
```

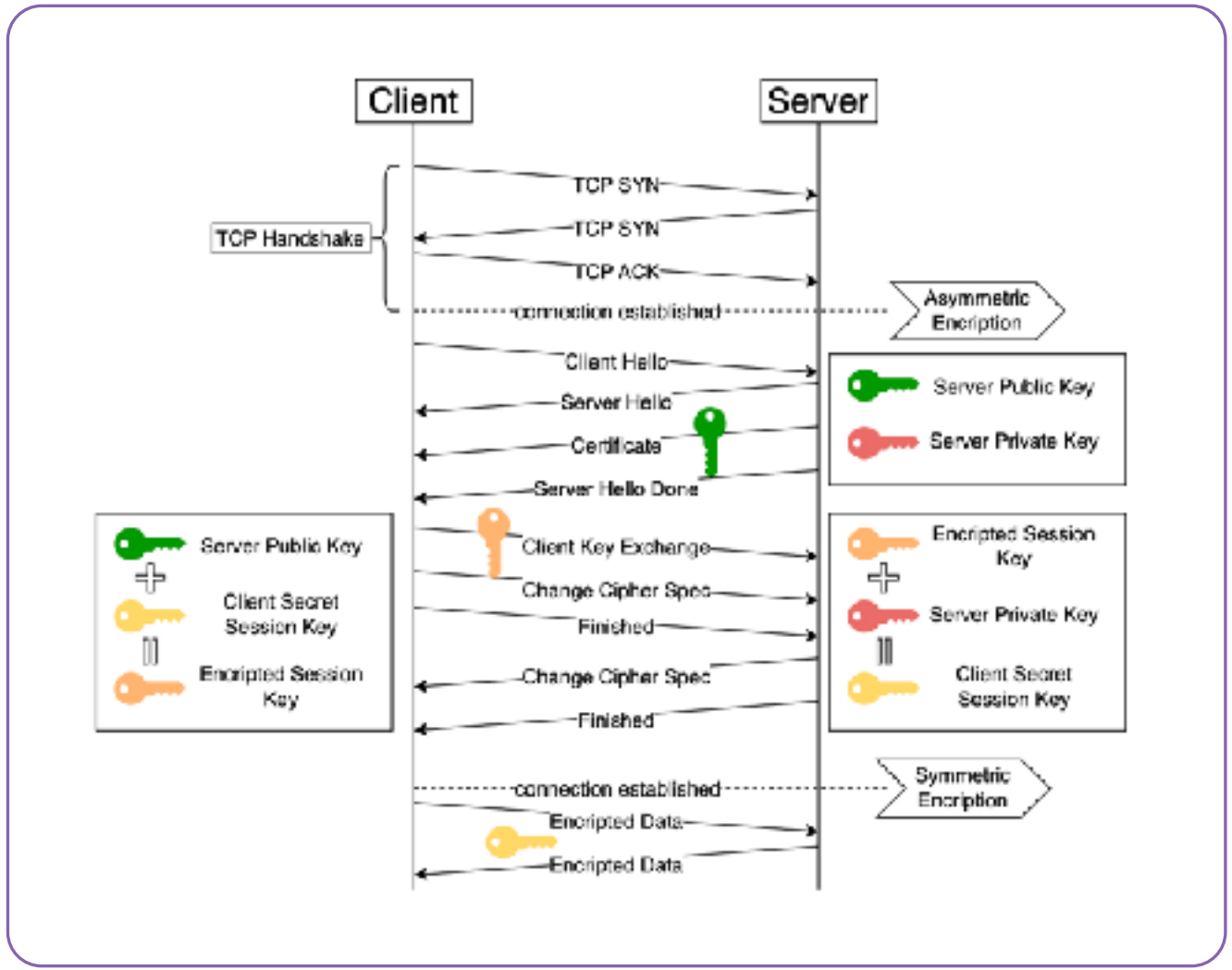
# HTTPS

- Supports TLS / SSL encryption methods
- Protects against man-in-the-middle attacks
- Requires a trusted third party to sign server-side digital certificates.
- Uses public and private keys to encrypt the data flow.



# TSL Handshaking

1. Establishing TCP connection
2. Client Hello: including the version of TLS it supports, a random number, and a list of cipher suites.
3. Server Hello: selects the version of TLS, a random number, and the cipher suite that will be used for the secure connection.
4. Certificate: The server sends its digital certificate to the client, which includes the server's public key.
5. Client Key Exchange: The client generates a secret key and encrypts it with the server's public key, then sends it to the server.
6. Change Cipher Spec: Both the client and server agree to use the newly established secret key for all future communication.

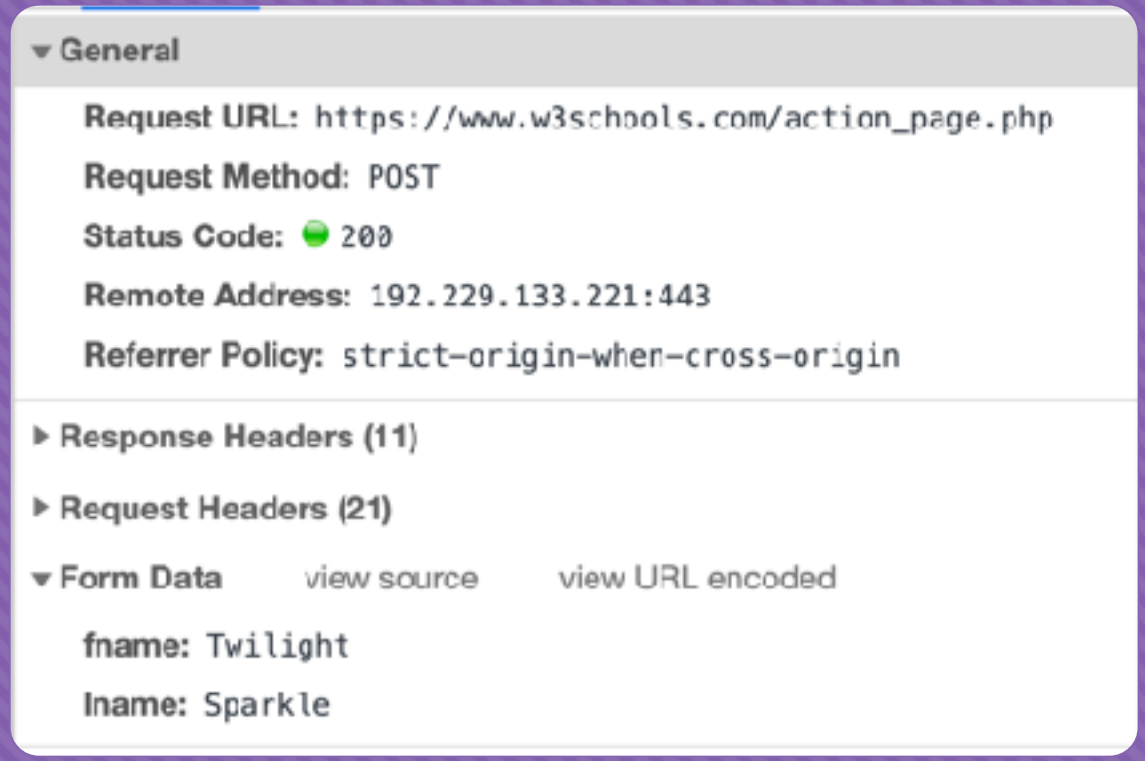


```html
<!DOCTYPE html>
<html>
<body>
  <form action="/action_page.php" method="POST">

    <label for="fname">First name:</label><br>
    <input type="text"
           id="fname"
           name="fname"
           value="Twilight"><br>

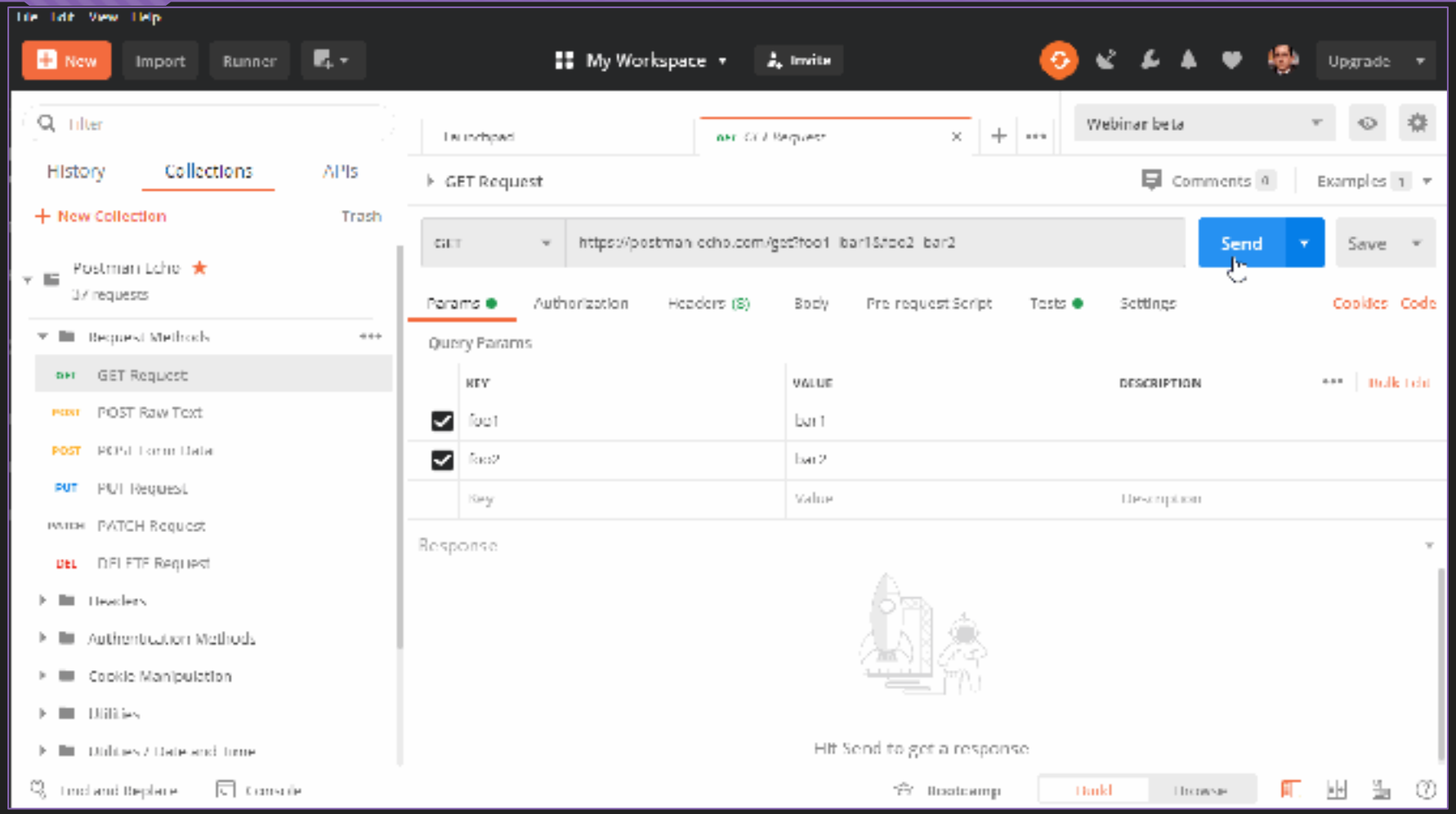
    <label for="lname">Last name:</label><br>
    <input type="text"
           id="lname"
           name="lname"
           value="Sparkle"><br><br>

    <input type="submit" value="Submit">
  </form>
</body>
</html>
```




# HTTP. Example

# Postman – tool for work with HTTP

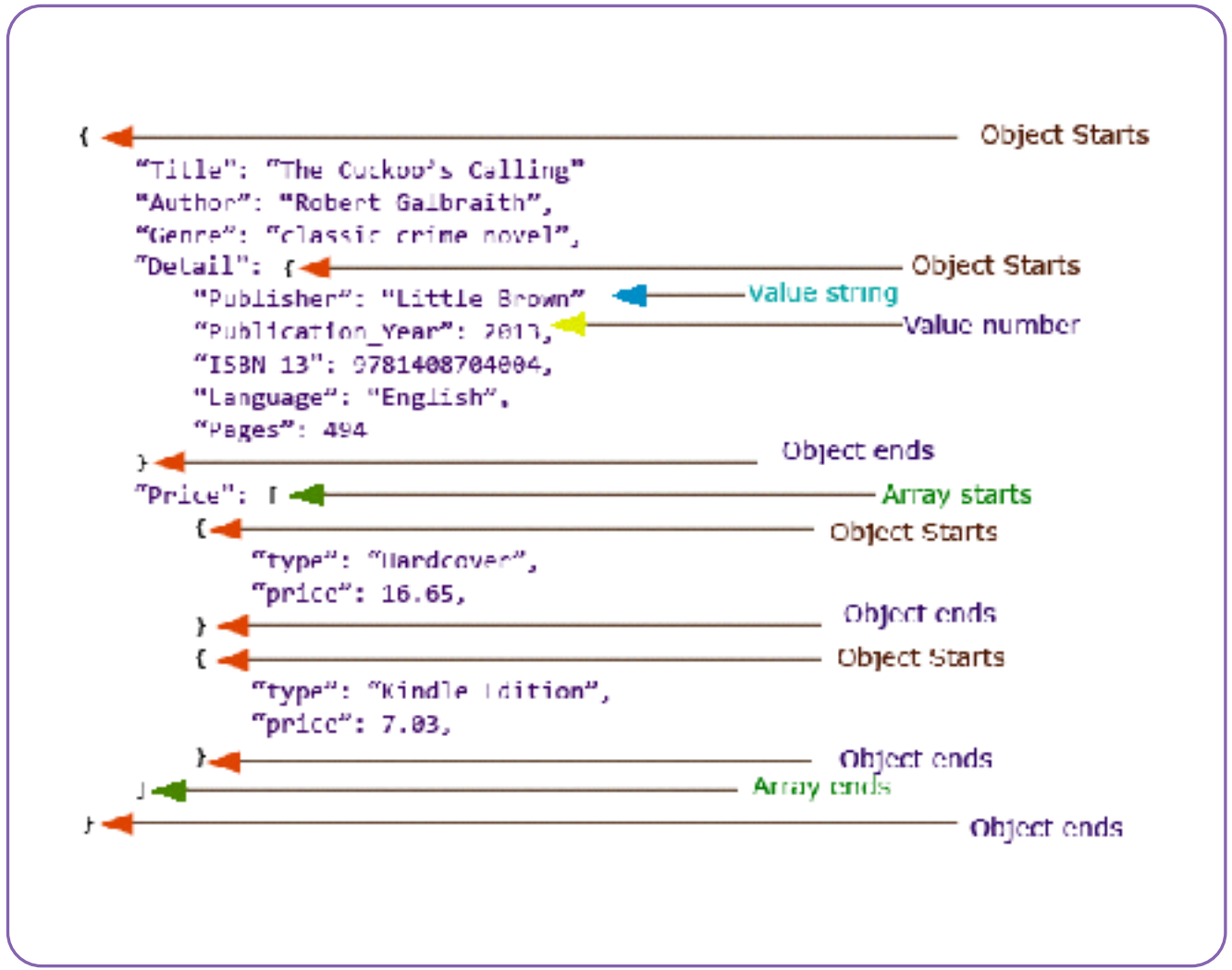


# JSON

# JSON

JSON data types:

- Number
- String
- Boolean
- Array
- Object
- null



# Dynamic web application

# Static web page




man7.org/linux/man-pages/man1/ls.1.html

man7.org > Linux > man-pages — Linux/UNIX system programming training

## ls(1) — Linux manual page

NAME | SYNOPSIS | DESCRIPTION | AUTHOR | REPORTING BUGS | COPYRIGHT | SEE ALSO | COLOPHON

Search online pages

**LS(1)** **User Commands** **LS(1)**

**NAME** top

ls - list directory contents

**SYNOPSIS** top

**ls** [*OPTION*]... [*FILE*]...

**DESCRIPTION** top

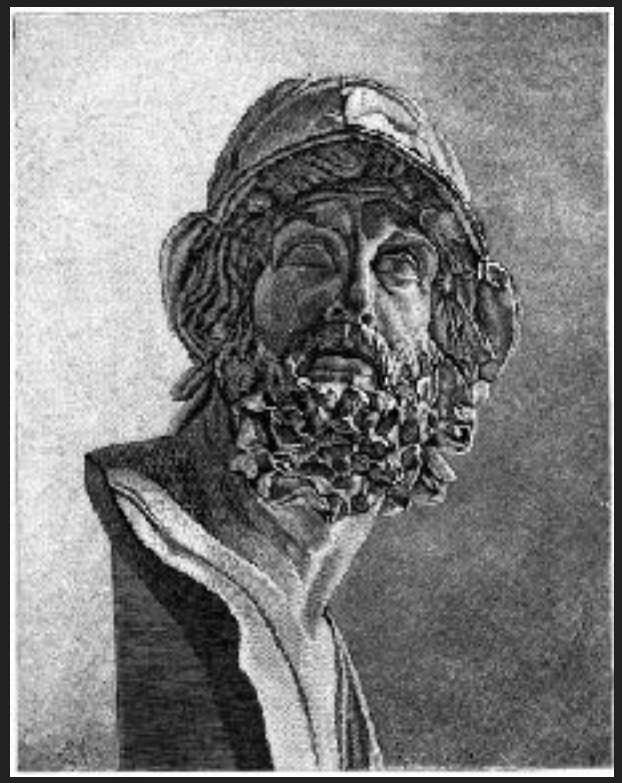
List information about the FILEs (the current directory by default). Sort entries alphabetically if none of **-cftuvSUX** nor **--sort** is specified.

# AJAX ?




Bleach ?

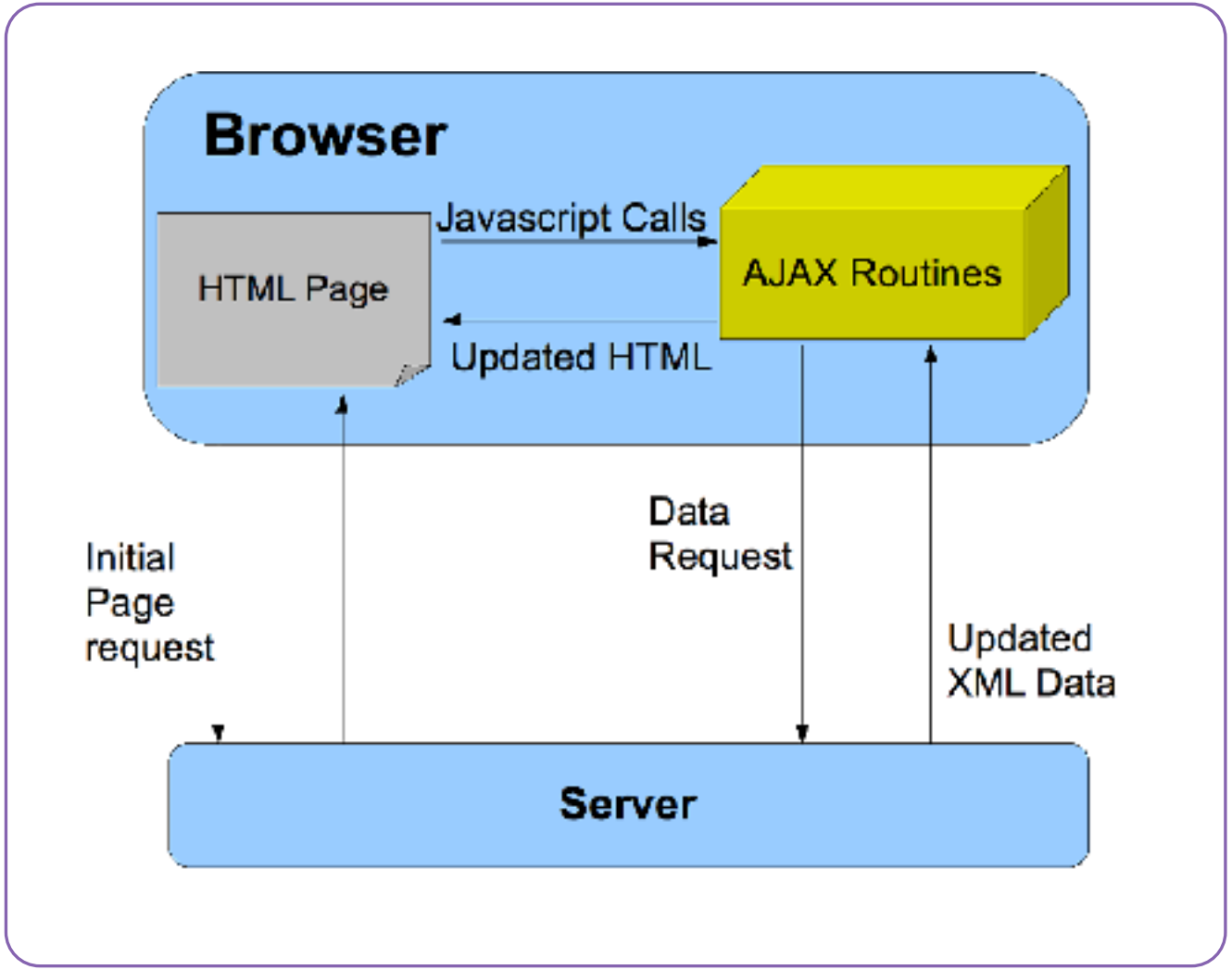
Football club ?



Greek hero?

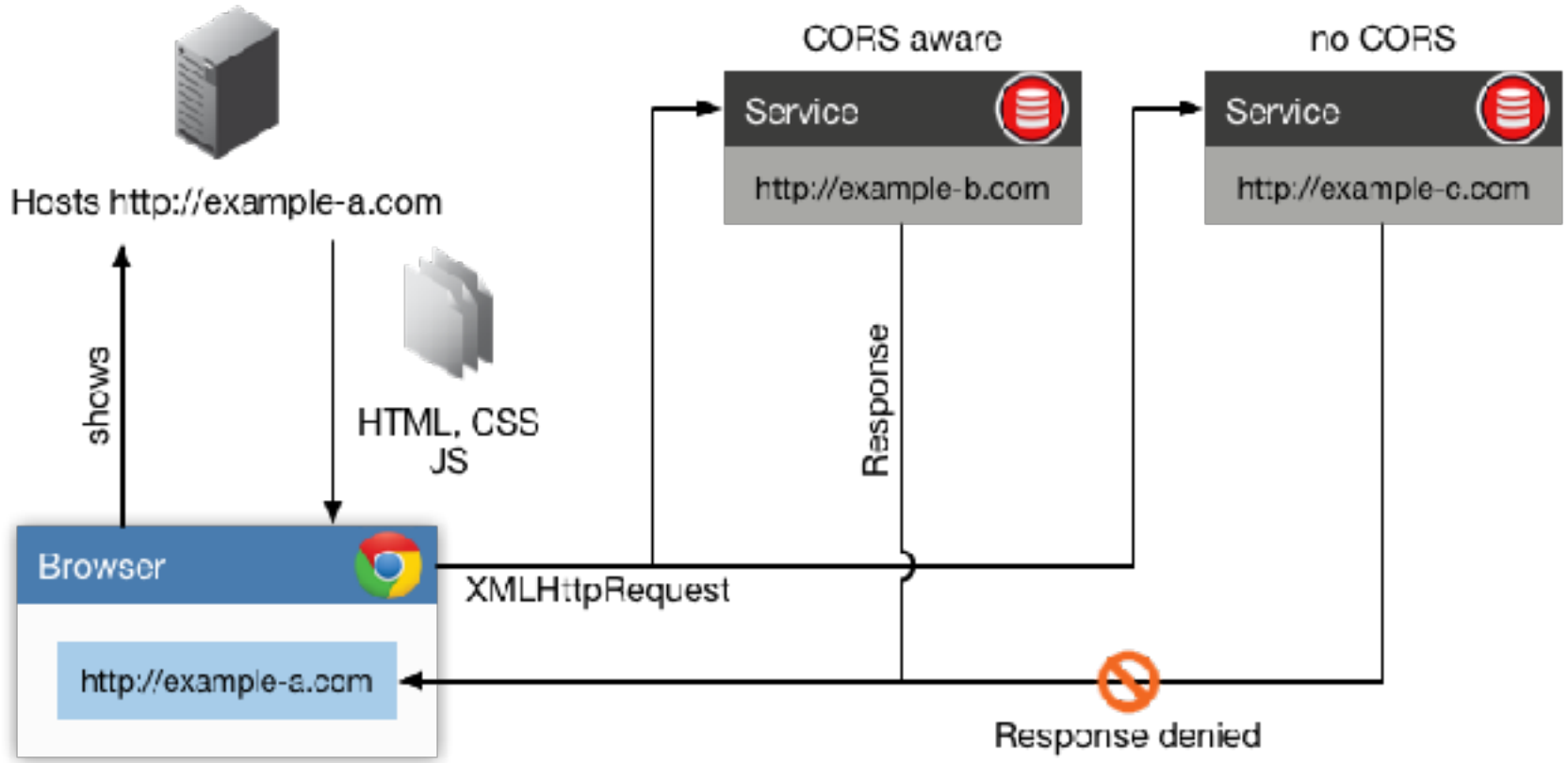
# AJAX - asynchronous JavaScript and XML

Ajax is "the method of exchanging data with a server and updating parts of a web page – without reloading the entire page."



# Security

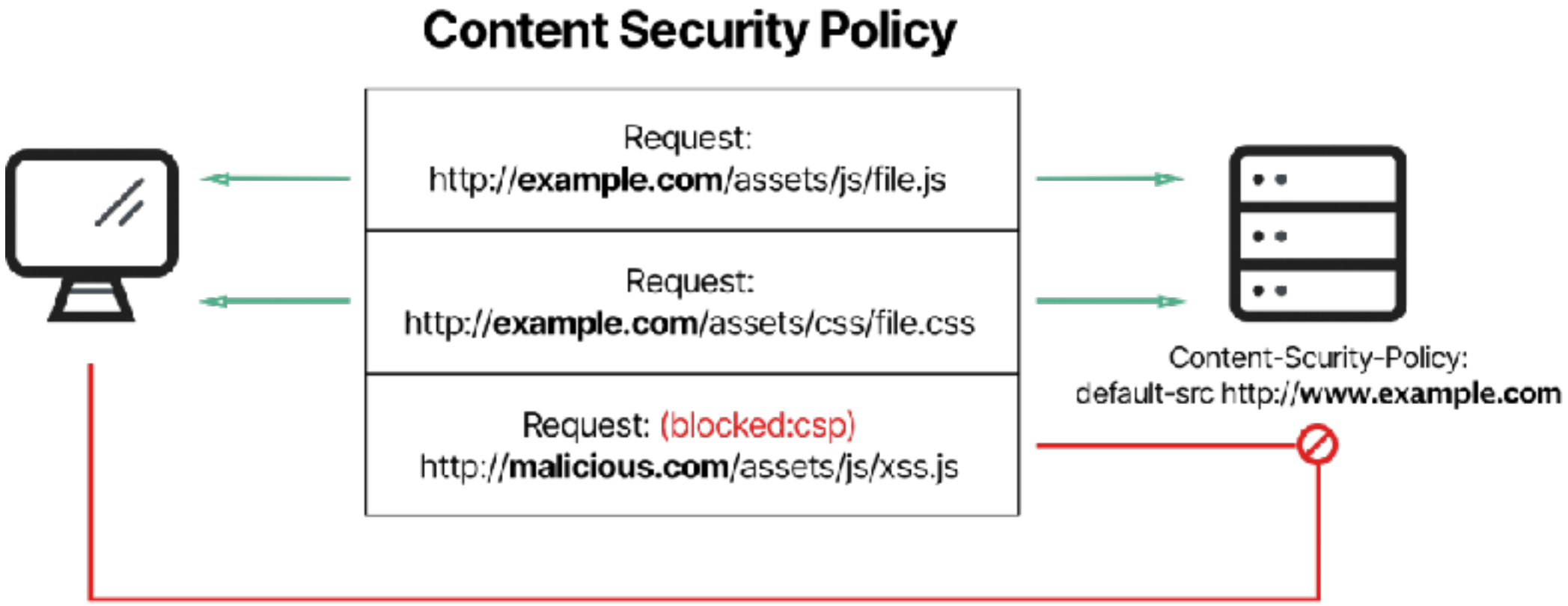
# Cross-Origin Resource Sharing (CORS)



# AJAX. CORS. Example
# Compare with "http://www.example.com/dir/page.html"

| Compared URL | Outcome | Reason |
|---|---|---|
| **http://www.example.com**/dir/page2.html | Success | Same protocol, host and port |
| **http://www.example.com**/dir2/other.html | Success | Same protocol, host and port |
| **http:**//username:password@**www.example.com**/dir2/other.html | Success | Same protocol, host and port |
| http://www.example.com:**81**/dir/other.html | Failure | Same protocol and host but different port |
| **https:**//www.example.com/dir/other.html | Failure | Different protocol |
| http://**en.example.com**/dir/other.html | Failure | Different host |
| http://**example.com**/dir/other.html | Failure | Different host (exact match required) |
| http://**v2.www.example.com**/dir/other.html | Failure | Different host (exact match required) |
| http://www.example.com:**80**/dir/other.html | Depends | Port explicit. Depends on implementation in browser |

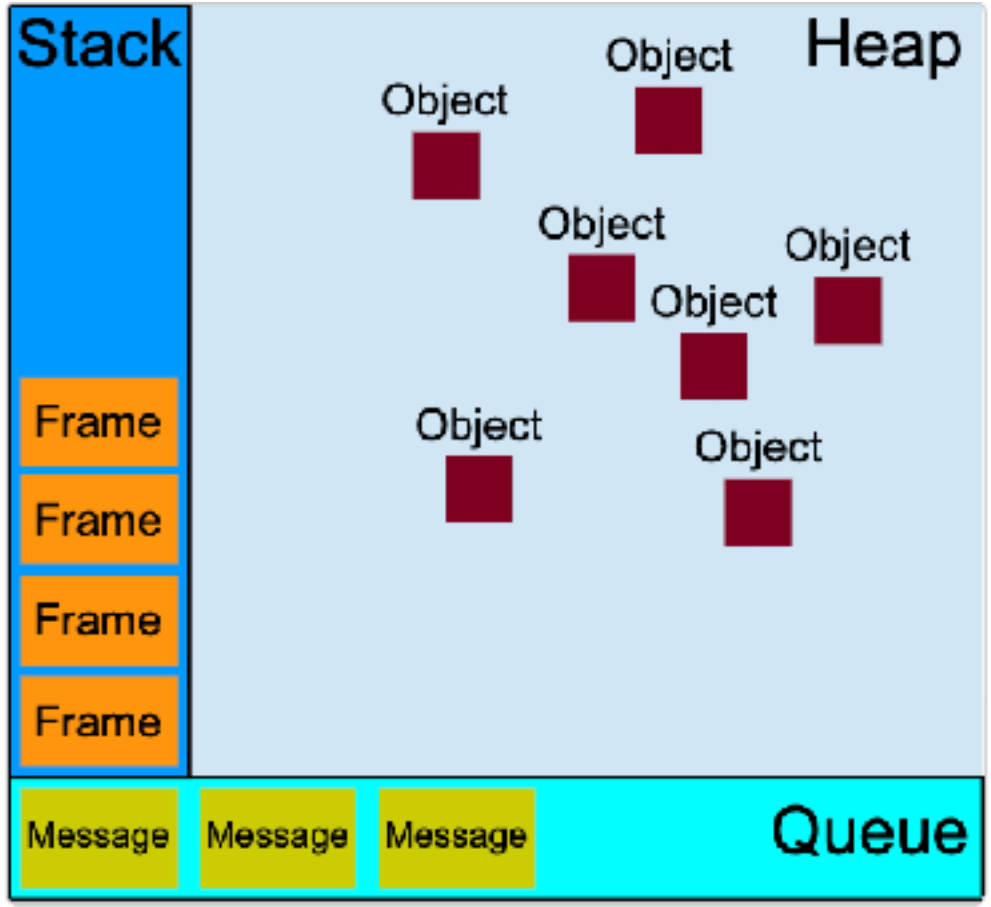
# Content Security Policy (CSP)



# Event Loop

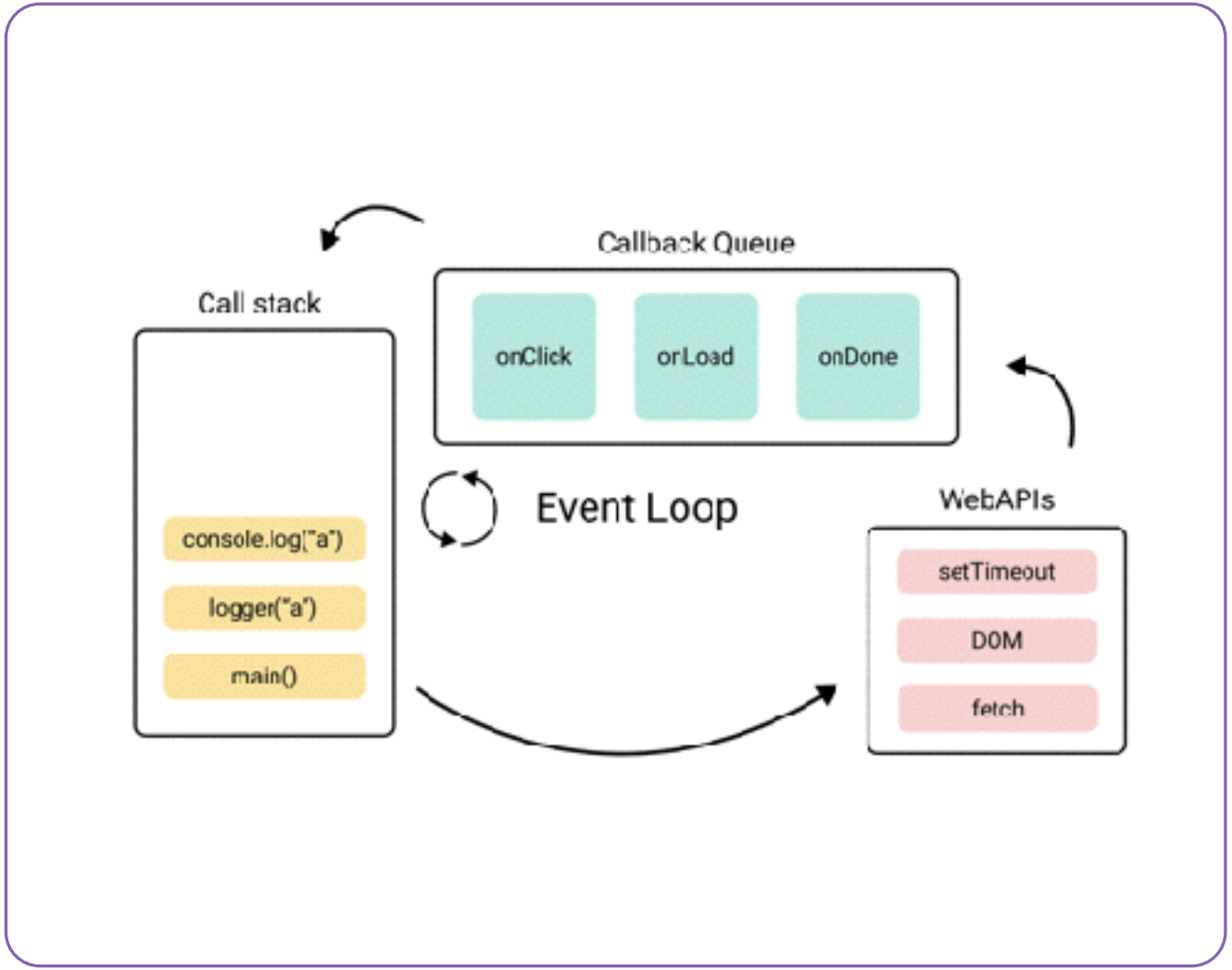
# JS Runtime

- Function calls form a stack of *frames*
- Objects are allocated in a heap
- A JavaScript runtime uses a message queue, which is a list of messages to be processed. Each message has an associated function that gets called to handle the message.



# Event Loop Visualization

- JS runs code in a single thread. Runtime finish every message one my one.
- Messages may be added through different API:
  - DOM events
  - Network requests
  - setTimeout



# Adding Messages

- ***setTimeout*** sets the message in the end of the queue after a minimum timeout (second argument)
- Zero timeout means that the message will be placed in the end of the current queue

```
> (() => {
    console.log("1");

    setTimeout(() => {
    console.log("timeout 1");
    }); // has a default time value of 0

    console.log("2");

    setTimeout(() => {
    console.log("timeout 2");
    }, 0);

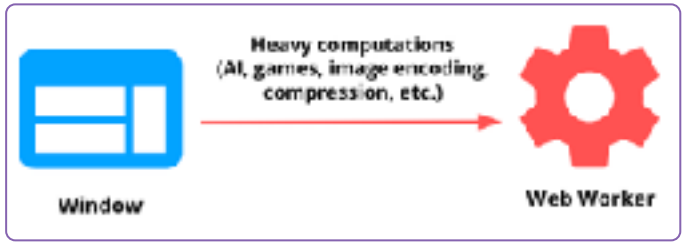
    console.log("3");

    setTimeout(() => {
    console.log("timeout 3");
    }, 1000);
})()
1
2
3
< undefined
timeout 1
timeout 2
timeout 3
```

# Several Runtimes Communication

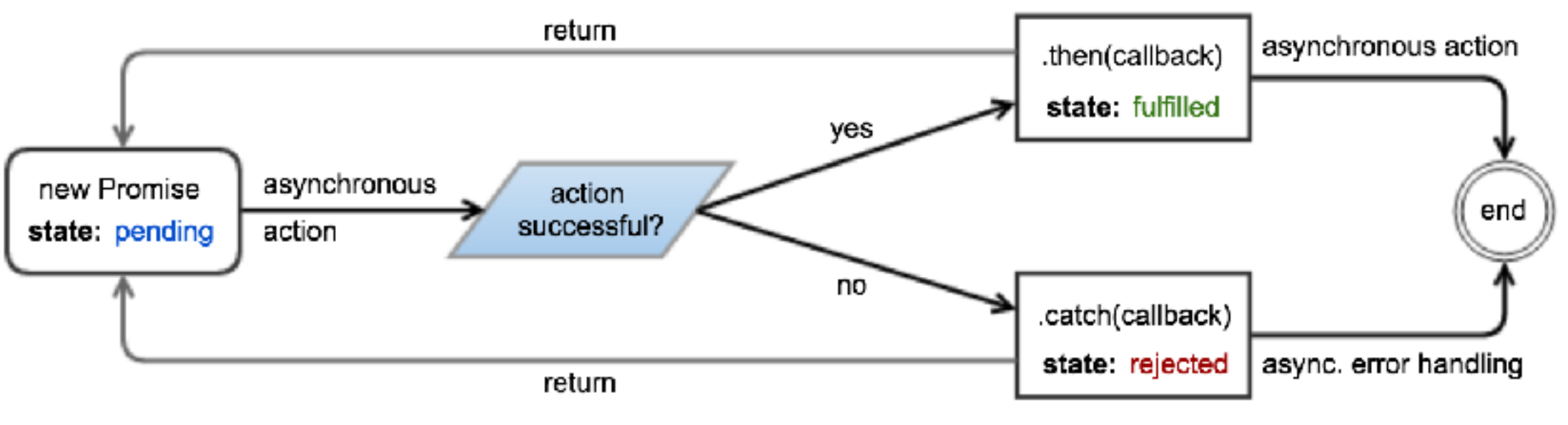
- A web worker or a cross-origin iframe has its own stack, heap, and message queue
- Two distinct runtimes can only communicate through sending messages via the ***postMessage*** method



# Promises

# AJAX. Promises



# Promises

| Name | Description |
| --- | --- |
| Promise.all() | The Promise.all(iterable) method returns a single Promise that resolves when all of the promises in the iterable argument have resolved or when the iterable argument contains no promises. It rejects with the reason of the first promise that rejects. |
| Promise.race() | The Promise.race(iterable) method returns a Promise that resolves or rejects as soon as one of the promises in the iterable resolves or rejects, with the value or reason from that promise. |
| Promise.resolve() | The Promise.resolve(value) method returns a Promise object that is resolved with the given value. |
| Promise.reject() | The Promise.reject(reason) method returns a Promise object that is rejected with the given reason. |
| Promise.prototype.then() | The then() method returns a Promise. It takes up to two arguments: callback functions for the success and failure cases of the Promise |
| Promise.prototype.catch() | he catch() method returns a Promise and deals with rejected cases only. |
| Promise.prototype.finally() | The finally() method returns a Promise. When the promise is settled, whether fulfilled or rejected, the specified callback function is executed |

# Async Promises Execution

- Promises executes in the end of each stack
- Callback in Promise constructor is called synchronously

```
> console.log("1");

  Promise.resolve()
      .then(() =>
          console.log("promise 1"),
      );

  const promise = new Promise(
      (resolve, reject) => reject(),
  );

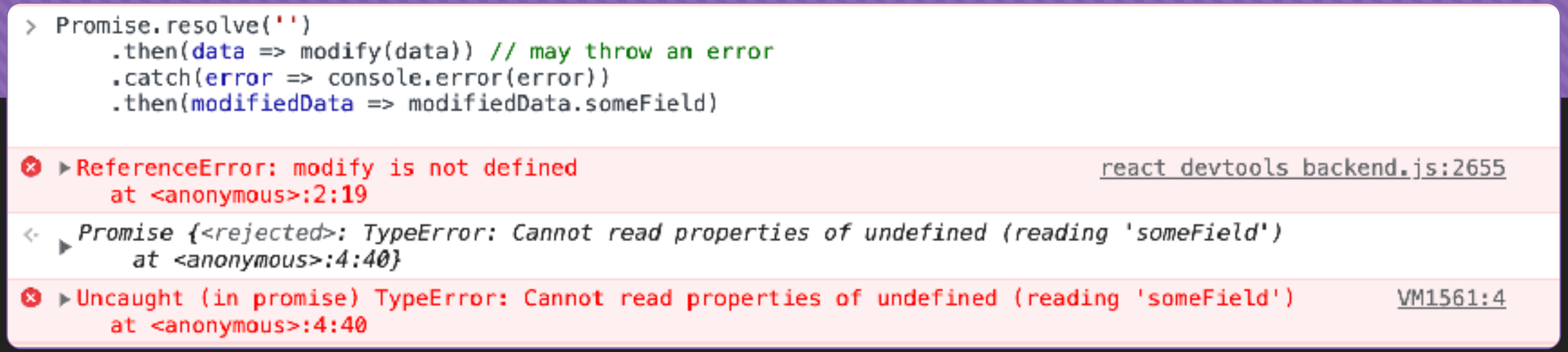
  promise
      .then(() =>
          console.log("never")
      )
      .catch(() =>
          console.log("promise 2")
      );

  console.log("2");
  1
  2
  promise 1
  promise 2
<- undefined
```

```
> function a() {
      console.log('a 1');
      Promise.resolve()
          .then(() =>
              console.log("promise"),
          );
      console.log('a 2');
  }

  function b() {
      console.log('b 1');
      a();
      console.log('b 2');
  }

  b();
  b 1
  a 1
  a 2
  b 2
  promise
<- undefined
```




# Promises. Error handling. Wrong approach

```
> Promise.resolve(data)
      .then(someHandler1) // works fine
      .then(someHandler2) // throws error
      .then(someHandler3) // wont be called
      .catch(console.error);
```

Promises. Error handling.
Catch at the end of the chain

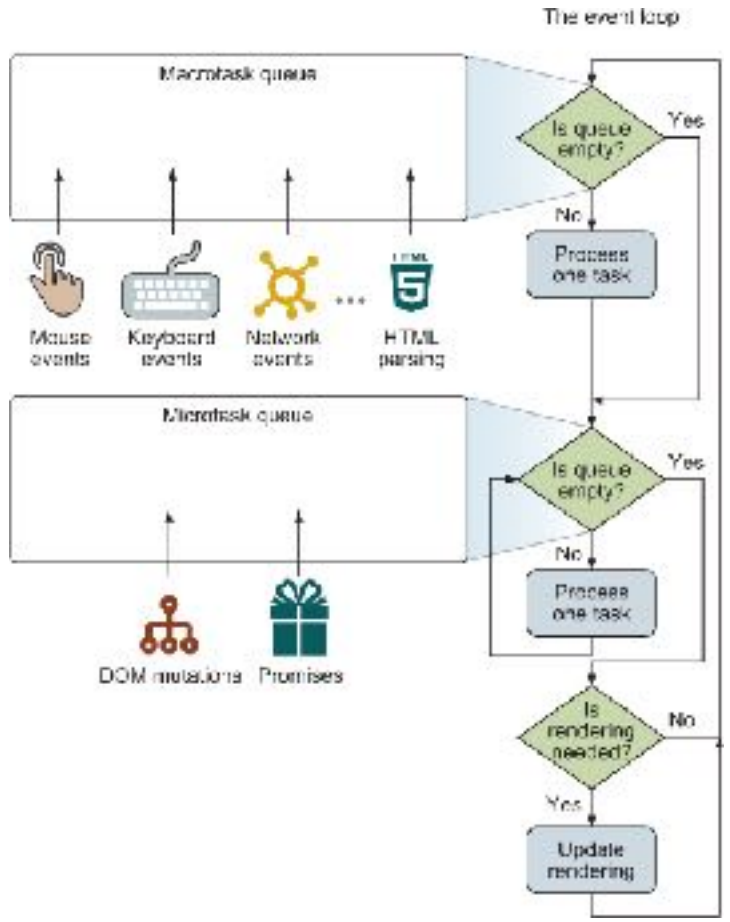



# Promises. Error handling. Rethrowing

# Microtasks and Macrotasks

# Visualization

- Each task in Macrotask queue has synchronous call stack and asynchronous microtask queue
- Macrotasks may be added through different API
- Microtasks may be added as a DOM mutations or Promises



# Example

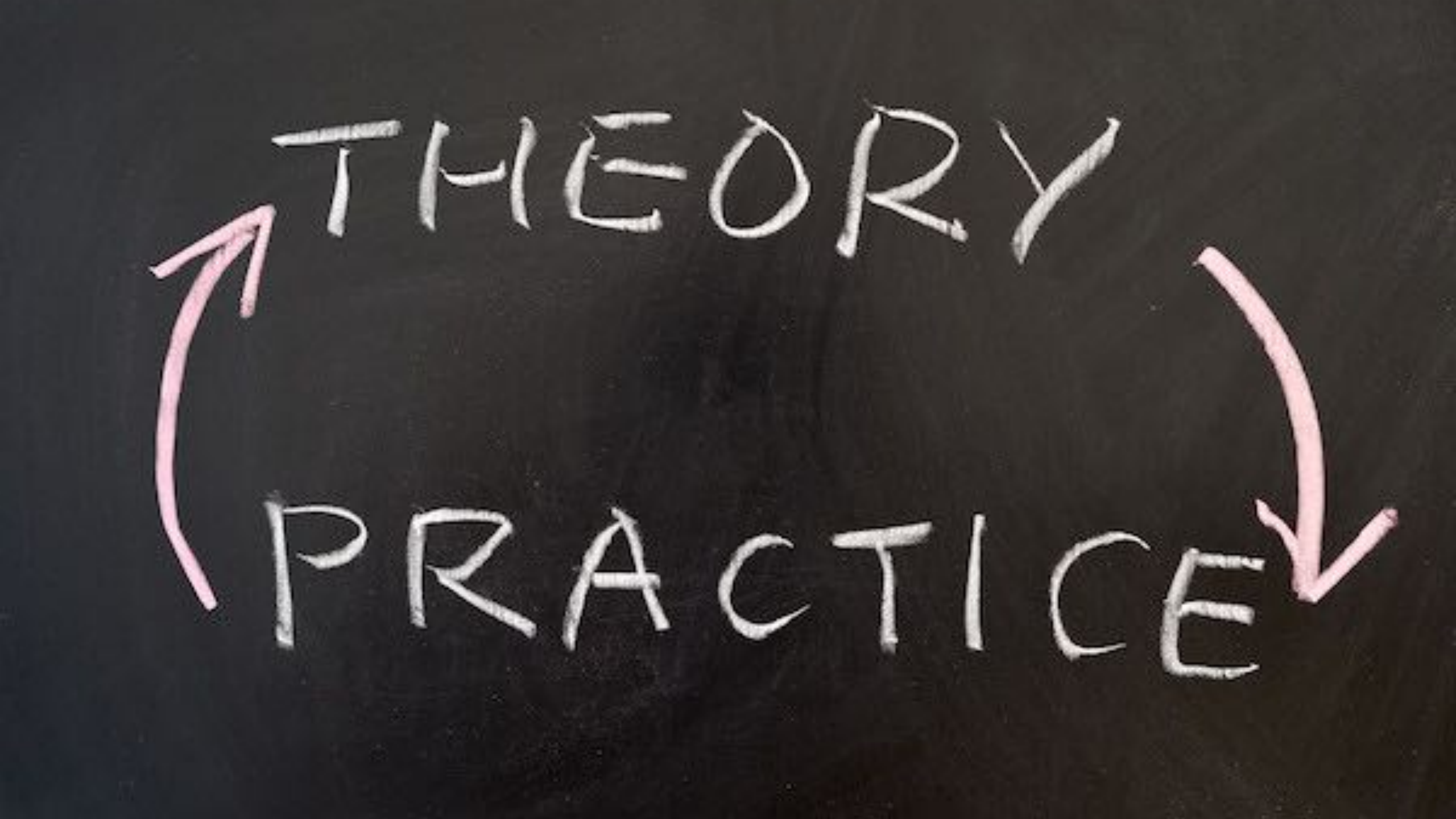
- Macrotask 1:
  - Sync execution: 1, 5, 7
  - Microtasks: 6
- Macrotask 2:
  - Sync execution: 4
  - Microtask: 3
- Macrotask 3:
  - Sync execution: 2

```
> (() => {
      console.log(1);

      setTimeout(() => console.log(2), 1);
      setTimeout(() => {
          Promise.resolve().then(() => console.log(3));
          console.log(4);
      }, 0);

      new Promise(function(resolve){
          console.log(5);
          resolve();
      }).then(function(){
          console.log(6);
      });

      console.log(7);
  })();
  1
  5
  7
  6
< undefined
  4
  3
  2
```


THEORY
PRACTICE

# AJAX. Pure JS approach

```
<div id="demo">
<button type="button" onclick="loadXMLDoc()">Change Content</button>
</div>

<script>
  function loadXMLDoc() {
    let xhttp = new XMLHttpRequest();

    // callback should be regular function
    xhttp.onreadystatechange = function() {
      if (this.readyState == 4 && this.status == 200) {
        document.getElementById("demo").innerHTML = this.responseText;
      }
    };

    xhttp.open(
      "GET",                  // method
      "xmlhttp_info.txt",     // url
      true,                   // async
    );

    xhttp.send();
  }
</script>
```

# AJAX. Pure JS approach. XMLHttpRequest States

- UNSENT – Client has been created. `open()` method was not called yet
- OPENED – `open()` method was called.
- HEADERS_RECEIVED – `send()` method was called. Headers and status are available already
- LOADING – Downloading is in progress. `responseText` holds partial data
- DONE – The operation is complete.

```
var xhr = new XMLHttpRequest();
console.log('UNSENT', xhr.readyState); // readyState will be 0

xhr.open('GET', '/api', true);
console.log('OPENED', xhr.readyState); // readyState will be 1

xhr.onprogress = function () {
    console.log('LOADING', xhr.readyState); // readyState will be 3
};

xhr.onload = function () {
    console.log('DONE', xhr.readyState); // readyState will be 4
};

xhr.send(null);
```

# AJAX
# Pure JS approach
# Complex example

- Task: request user from one source and then request the github data of that user

```
function loadXMLDoc() {
    const xhr = new XMLHttpRequest();
    xhr.open('GET', '/some/url/to/users.json', true);

    xhr.onload = function() {
      if (this.status === 200) {
        // side effects
        console.log(response)
        const user = JSON.parse(response);
        console.log(user);

        const secondXhr = new XMLHttpRequest();
        secondXhr.open('GET',`https://api.github.com/users/${user.name}`, true);

        secondXhr.onload = function() {
          if (this.status === 200) {
            console.log('github user', this.response);
          }
        };

        secondXhr.send();
      }
    };

    xhr.send();
}
```





# Warning: Hell is possible

# AJAX. Promises approach

```
function httpGet(url) {
    return new Promise((resulve, reject) => {
        let xhr = new XMLHttpRequest();
        xhr.open('GET', url, true);

        xhr.onload = function() {
            if (this.status === 200) {
                resolve(this.response);
            } else {
                let error = new Error(this.statusText);
                error.code = this.status;
                reject(error);
            }
        };

        xhr.onerror = function() {
            reject(new Error('Network Error'));
        };

        xhr.send();
    });
}
```

# AJAX. Promises approach

```
// usage
httpGet('/some/url/to/users.json')
    .then(
        response => alert(`Fulfilled: ${response}`),
        error => alert(`Rejected: ${error}`),
    );

// or
httpGet('/some/url/to/users.json')
    .then(response => alert(`Fulfilled: ${response}`));
    .catch(error => alert(`Rejected: ${error}`));
```

# AJAX. Promises approach. Complex Example

- Same task: request user from one source and then request the github data of that user

```
httpGet('some/url/to/users.json')
    .then(response => {
        console.log(response) // side effects

        // returns string
        return JSON.parse(response);
    })
    .then(user => {
        console.log(user);

        // returns promise
        return httpGet(`https://api.github.com/users/${user.name}`);
    })
    .then(githubUser => {
        console.log(githubUser);
    })
```


Callback Hell
Promise Hell
Warning: Hell is still possible…

# Existing solution

- Fetch API – built in browser API for accessing and manipulating parts of the protocol, such as requests and responses
- NPM packages:
  - Axios
  - Superagent
  - Got
  - etc



```
function postData(url = "", data = {}) {
    // Default options are marked with *
    return fetch(url, {
        method: "POST", // *GET, POST, PUT, DELETE, etc.
        mode: "cors", // no-cors, *cors, same-origin
        cache: "no-cache", // *default, no-cache, reload, force-cache, only-if-cached
        credentials: "same-origin", // include, *same-origin, omit
        headers: {
          "Content-Type": "application/json",
          // 'Content-Type': 'application/x-www-form-urlencoded',
        },
        redirect: "follow", // manual, *follow, error
        referrerPolicy: "no-referrer", // no-referrer, *no-referrer-when-downgrade, origin, origin-when-
cross-origin, same-origin, strict-origin, strict-origin-when-cross-origin, unsafe-url
        body: JSON.stringify(data), // body data type must match "Content-Type" header
      })
        .then(response => response.json()) // parses JSON response into native JavaScript objects
        .catch(error => {
            console.error(error);
            return null; // instead of expected data
        });
}

postData("https://example.com/answer", { answer: 42 }).then((data) => {
  console.log(data); // JSON data parsed by `data.json()` call or null in case of error
});
```

# AJAX. Fetch



{Syntactic Sugar}

# Async & Await

*"async and await make promises easier to write"*

- **async** makes a function return a Promise
- **await** makes a function wait for a Promise

```
async function myDisplay() {
  let promise = new Promise(function(resolve, reject) {
    resolve("Some text");
  });

  document.getElementById("demo").innerHTML = await promise;
}

myDisplay();
```

# Async & Await. Error handling

- Rules are the same as for regular code flow

```
> const getFileLenght = async () => {
    try {
        return (await readFileContent() ).lendth;
    } catch (error) {
        console.error('Error occured:', error);
    } finally {
        console.log('finally');
    }
}

getFileLenght();
▸ Error occured: ReferenceError: readFileContent is not defined
    at getFileLenght (<anonymous>:3:9)
    at <anonymous>:11:1
finally
▸ Promise {<fulfilled>: undefined}
```

# Async & Await. Bad practices

- Never await in return. It is redundant
- If there are no awaits, you should not make the function async unless you want it to be a promise

```
async httpGet(url) {
    return await fetch(url);
}
```

Bad

```
let state = 0;

const asyncLog = async () => {
    await Promise.resolve();
    console.log('async state: ', state)
};
const modifyState = () => state++;

asyncLog();
modifyState();
```

```
async state:  1
```

Situationally

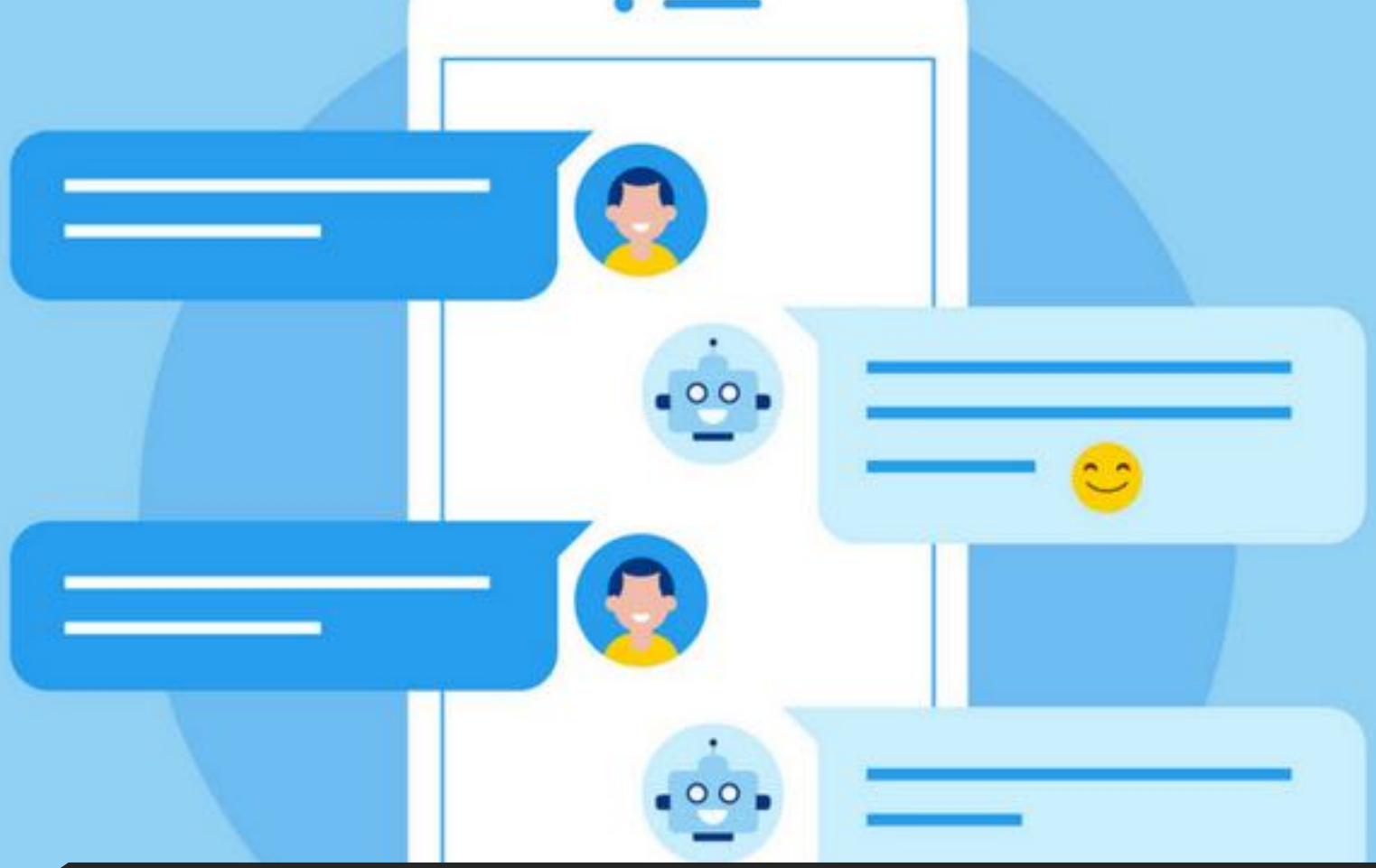
```
> const simplePromise = async () => 1;
  simplePromise();
< ▶ Promise {<fulfilled>: 1}
> const regularPromise = () => Promise.resolve(1);
  regularPromise();
< ▶ Promise {<fulfilled>: 1}
```

# AJAX. Complex Example. Again... =)

- Same task: request user from one source and then request the github data of that user

```
> async function executeTask() {
      const response = await httpGet('some/url/to/users.json');
      console.log(response);
      const user = JSON.parse(response);
      console.log(user);

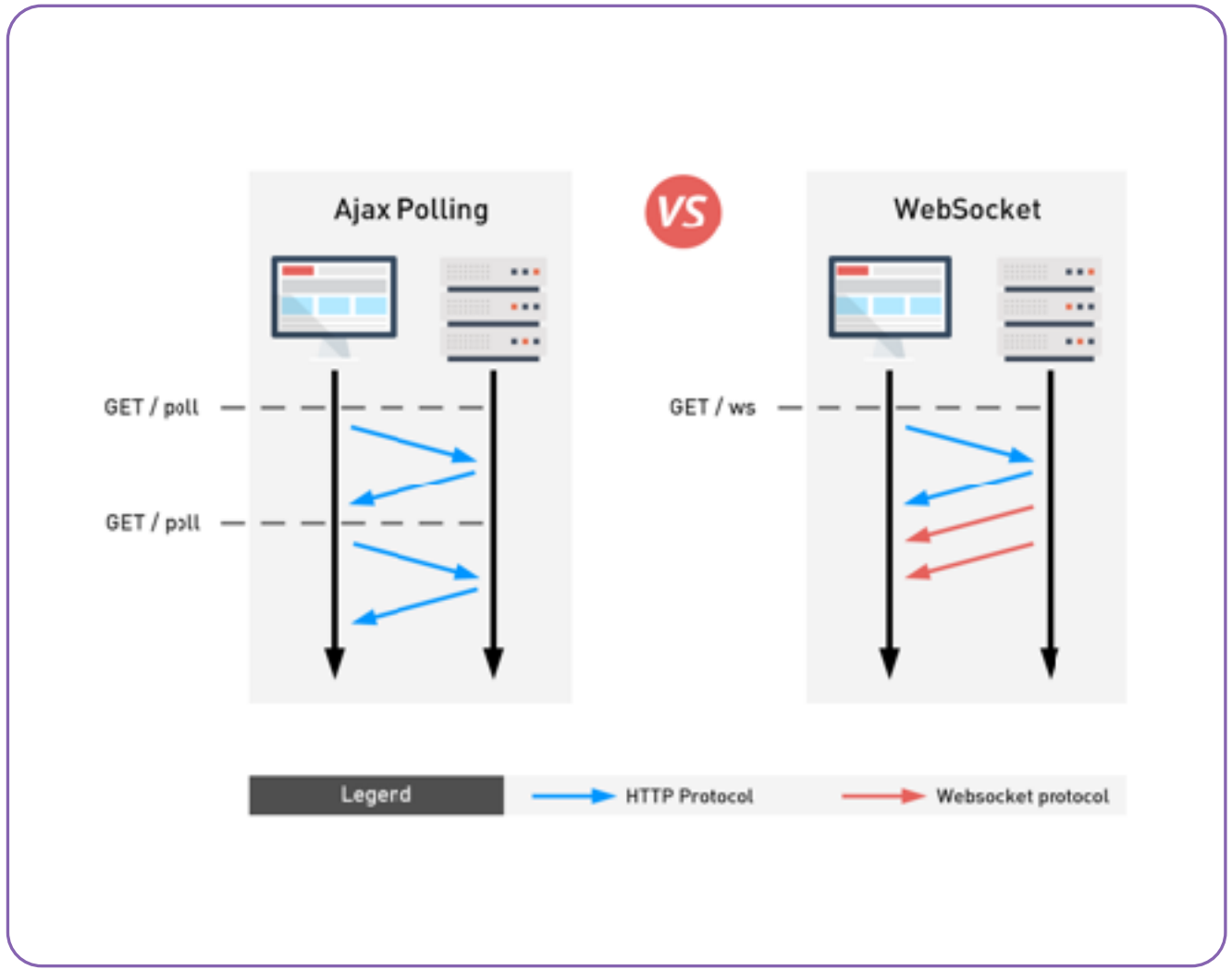
      const githubUser = await httpGet(
          `https://api.github.com/users/${user.name}`,
      );
      console.log(githubUser);
  }
```



# How do online chats work then?

# WebSocket

WebSocket is a computer communications protocol, providing full-duplex communication channels over a single TCP connection.



# Q&A

